School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory Assignment #5: Building a Web API with Express

Objectives:

- To build an Express Web API, routes that return JSON over HTTP, not an HTML website.
- To serve data from the bookstore database through API routes using mysql2.
- To read route parameters, query strings and request bodies, and use status codes correctly.
- To call your API from a simple web page with fetch(), keeping the two halves, Web API and website clearly separated.

Scenario: build a small Web API over the bookstore database from Lab 2, already filled with data in Lab 3. “Web API” is precise here on purpose: every route below returns JSON for a program to consume, not an HTML page for a person to read that distinction is what Lab 5 introduced. Your Express server will expose routes that read and create data, and a simple static page will call them the same way, as a separate program, over HTTP. Build on your Lab 4 mysql2 code for the queries.

```
books(     id, title, author, price, stock )
customers( id, name, email )
orders(    id, customer_id, book_id, quantity )

-- customer_id -> customers(id)
-- book_id     -> books(id)
```

Set up the project (npm init -y, then npm install express mysql2) and write your server in app.js. Use the promise API for queries and pass every value from req.params, req.query or req.body through a ? placeholder. Remember that express.json() is middleware: without app.use(express.json()) near the top of your file, req.body will be undefined in every route that needs it. The skeleton below gets you started. Submit your .js file(s) and any index.html, together with screenshots of the browser (or REST client) output for the tasks that return data. This assignment is marked out of 12.

```
const express = require('express');
const mysql = require('mysql2/promise');
const app = express();
app.use(express.json());

// ... your routes here ...

app.listen(3000, () => console.log('http://localhost:3000'));
```

How to run: npm install (installs express and mysql2), then node app.js and open http://localhost:3000. Restart node app.js after every code change.
A browser address bar can only send GET requests — to test POST /books (Task 5) you need a REST client such as Postman, Insomnia, the Thunder Client VS Code extension, or curl:

```
curl -X POST http://localhost:3000/books \
  -H "Content-Type: application/json" \
  -d '{"title":"New Book","author":"A. Author","price":499,"stock":10}'

# expect: {"id": 6}  with status 201 Created
```

1. Set up Express. Create the project, install express and mysql2, and write a GET / route that returns a welcome message. Start the server and show it working in your browser (screenshot). **(1 points)**
2. List books. Write GET /books that queries the database and returns all books as JSON. Include a screenshot of the JSON in your browser. **(1.5 points)**
3. One book by id. Write GET /books/:id that returns a single book using req.params and a ? placeholder, and replies with status 404 if no book has that id. **(1.5 points)**
4. Filter with a query string. Extend GET /books to accept an optional ?author=... query string (req.query) and, when present, return only that author's books using a ? placeholder. **(1.5 points)**
5. Create a book. Add app.use(express.json()) and write POST /books that reads the new book from req.body, inserts it with ? placeholders, and replies with status 201 and the new id. **(2 points)**
6. Join from the API. Write GET /orders/details that returns, for every order, the customer's name, the book title and the quantity (an INNER JOIN) as JSON. Include a screenshot. **(1.5 points)**
7. Handle errors. Wrap each route handler in try / catch and return status 500 with a JSON error message on failure. In one sentence, explain why a route handler needs its own error handling. **(1.5 points)**
8. Give it a face. Serve a static public/index.html with app.use(express.static('public')) that uses fetch('/books') to list the books on the page. Include a screenshot of the page. **(1.5 points)**